

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
TP
Versione 1.3



Data: 21/12/2025

Progetto: MediBook	Versione: 1.3
Documento: TP	Data: 21/12/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Christian Casale
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.1	Aggiunta template e inizio stesura Suspension and resumption	Samuele Colucci
21/12/2025	1.2	Aggiunta test case	Samuele Colucci
21/12/2025	1.3	Revisione e correzione documento	Christian Casale

Contents

1.	Introduction	4
2.	Relationship to other documents	4
3.	System overview	4
4.	Features to be tested/not to be tested	4
5.	Pass/Fail criteria	4
6.	Approach.....	5
6.1	Testing di unità	5
6.2	Testing di integrazione	5
6.3	Testing di sistema	5
7.	Suspension and resumption	5
7.1	Criteri di Sospensione	5
7.2	Criteri di Ripresa	6
8.	Testing materials (hardware/software requirements)	6
9.	Test cases	6
9.1	Funzionalità: Autenticazione (UC1)	6
9.2	Funzionalità: Prenotazione Visita (UC3)	7
9.3	Funzionalità: Registrazione Account (UC2)	8
9.4	Funzionalità: Visualizzazione Referto (UC6)	9
9.5	Funzionalità: Modifica Prenotazione – Segreteria (UC4)	10

1. Introduction

La presente sezione descrive gli obiettivi e l'estensione delle attività di test per il sistema MediBook. L'obiettivo è fornire un quadro di riferimento per la verifica della conformità del software rispetto ai requisiti specificati nel RAD e nell'SDD.

2. Relationship to other documents

Il piano di test fa riferimento ai seguenti documenti:

- RAD: Per la verifica dei requisiti funzionali (Casi d'Uso).
- ODD: Per la verifica dei vincoli sulle classi e sui dati (Invarianti OCL).

Naming Scheme: I casi di test saranno identificati univocamente con la stringa TC_<Funzione>_<Numero> (es. TC_PRE_1).

3. System overview

Il sistema oggetto di test è composto dai seguenti macro-componenti:

- Client Web (Presentation): Interfaccia sviluppata in ReactJS.
- Application Server (Business): Logica di controllo Java.
- Database (Persistence): MySQL Server.

Il focus dei test descritti in questo piano riguarda principalmente la logica di business e l'integrazione tra i sottosistemi Gestione Utenza e Gestione Appuntamenti.

4. Features to be tested/not to be tested

Funzionalità da testare:

- Autenticazione (Login/Logout).
- Registrazione Paziente.
- Prenotazione Visita (verifica disponibilità slot).
- Visualizzazione Referto.

Funzionalità non testate:

- Performance del database sotto stress estremo.
- Compatibilità con browser obsoleti (es. IE11).
- Funzionalità native di librerie di terze parti (Spring, React).

5. Pass/Fail criteria

Pass: L'output effettivo coincide con l'output atteso e non vengono sollevate eccezioni non gestite.

Fail: L'output differisce, il sistema va in crash, o viene mostrato un errore non user-friendly.

6. Approach

L'attività di testing del sistema MediBook è stata suddivisa in tre livelli logici sequenziali, partendo dalla verifica dei singoli componenti fino alla validazione del sistema completo.

6.1 Testing di unità

In questa fase vengono testate le singole classi in isolamento per garantirne il corretto funzionamento interno prima di integrarle con il resto del sistema.

Oggetto del test:

- Classi Entità (it.unisa.medibook.model): Verifica dei costruttori e dei metodi di accesso.
- Classi DAO (it.unisa.medibook.storage): Verifica delle query di inserimento, aggiornamento e lettura sul database.

Obiettivo: Assicurarsi che ogni modulo esegua correttamente il suo compito atomico e gestisca gli errori locali.

6.2 Testing di integrazione

Questa fase verifica la corretta comunicazione tra i moduli precedentemente testati, concentrandosi sullo scambio dati tra la logica applicativa e il livello di persistenza.

Oggetto del test: Interazione tra i Manager di business (it.unisa.medibook.business) e le classi DAO.

Obiettivo: Verificare che i dati passati dalla logica di business vengano correttamente mappati nel database e che le informazioni recuperate dal DB siano coerenti con quanto atteso dai metodi di gestione.

6.3 Testing di sistema

In questa fase finale viene verificato il comportamento globale del sistema attraverso l'interfaccia utente, simulando le azioni reali degli utenti finali.

Oggetto del test: Interfaccia Web (Frontend ReactJS) e API Backend.

Obiettivo: Validare la conformità del software rispetto ai requisiti funzionali descritti nel RAD (es. Casi d'uso di Login, Prenotazione, Gestione Referti), verificando che i flussi utente siano completi e privi di errori bloccanti.

7. Suspension and resumption

Questa sezione definisce i criteri per sospendere e riprendere le attività di test.

7.1 Criteri di Sospensione

	Ingegneria del Software	Pagina 5 di 10
--	-------------------------	----------------

Il testing sarà sospeso se si verifica una delle seguenti condizioni:

- Vengono scoperti bug critici (Blocking Bugs) che impediscono l'utilizzo delle funzioni principali (es. Login non funzionante).
- L'ambiente di test (Database o Server) non è disponibile o è instabile.
- Il codice rilasciato è incompleto rispetto alle specifiche previste per la fase corrente.

7.2 Criteri di Ripresa

Abbiamo previsto delle modifiche future al sistema dopo il rilascio.

Pertanto sarà necessario, dopo aver introdotto i cambiamenti, testare le nuove componenti: se esse introducono dei fault che impattano sulle componenti già esistenti, allora verranno testate nuovamente anche quest'ultime Testing materials (hardware/software requirements)

8. Testing materials (hardware/software requirements)

Per eseguire i test del sistema MediBook è sufficiente una sola postazione di lavoro (PC o Laptop) configurata come segue:

Hardware

- **Computer:** Un comune PC portatile (Windows o Mac) usato sia per scrivere il codice che per testarlo.

Software

- **Sistema Operativo:** Windows 10/11 o macOS.
- **Database:** MySQL Server (installato sul PC).
- **Ambiente Java:** JDK aggiornato (per far girare il backend).
- **Ambiente Web:** Node.js (per far partire il frontend React).
- **Browser:** Google Chrome.
- **Strumenti:** L'IDE usato per sviluppare (es. NetBeans o IntelliJ) per lanciare i test JUnit.

9. Test cases

In questa sezione vengono definiti i casi di test pianificati. Per garantire una copertura completa delle situazioni critiche, è stato utilizzato il metodo **Category Partition**, scomponendo gli input in categorie e scelte rappresentative.

9.1 Funzionalità: Autenticazione (UC1)

Verifica che il sistema gestisca correttamente l'accesso di utenti registrati e non.
Definizione dei Parametri e Categorie:

Parametro: Email [E]

- Email presente nel DB [propertyEmailOk]
- Email non presente [error]

Parametro: Password [P]

- Password corretta (corrispondente all'email) [propertyPassOk]
- Password errata [error]

ID Test Case	Combinazione	Descrizione Scenario	Esito Atteso
TC_AUC_1	E:2	Utente inserisce email inesistente.	Fail (Errore visualizzato)
TC_AUC_2	E:1, P:2	Utente inserisce email corretta ma password errata.	Fail (Errore credenziali)
TC_AUC_3	E:1, P:1	Utente inserisce credenziali corrette.	Pass (Redirect alla HomePage)

9.2 Funzionalità: Prenotazione Visita (UC3)

Verifica la logica di prenotazione, assicurando che non si possano creare sovrapposizioni o prenotazioni inconsistenti.

Definizione dei Parametri e Categorie:

Parametro: Data Visita [D]

- Data Futura (es. domani) [propertyDataOk]
- Data Passata o Odierna [error]

Parametro: Disponibilità Medico/Slot [S]

- Slot orario libero [propertySlotOk]
- Slot orario già occupato da altra visita [error]

Parametro: Stato Paziente [U]

- Paziente Attivo [propertyUserOk]
- Paziente Sospeso/Non Loggato [error]

ID Test Case	Combinazione	Descrizione Scenario	Esito Atteso
--------------	--------------	----------------------	--------------

ID Test Case	Combinazioni	Descrizione Scenario	Esito Atteso
TC_PRE_1	D:2	Tentativo di prenotazione nel passato.	Fail (Data non valida)
TC_PRE_2	D:1, S:2	Tentativo di prenotazione su orario occupato.	Fail (Slot non disponibile)
TC_PRE_3	D:1, S:1, U:2	Utente non loggato tenta di prenotare.	Fail (Richiesta Login)
TC_PRE_4	D:1, S:1, U:1	Utente attivo prenota in data valida e slot libero.	Pass (Prenotazione Confermata)

9.3 Funzionalità: Registrazione Account (UC2)

Si verifica che il sistema accetti la registrazione solo se tutti i campi obbligatori (Email, Password, Nome Utente, Codice Fiscale) rispettano i formati previsti e i vincoli di unicità nel database.

Parametro: Formato Email [F]

- $^{\wedge}[A-Za-z0-9+_{.-}] + @ [A-Za-z0-9.-] + \$$ Formato valido [propertyEmailOk]
- Formato non valido [error]

Parametro: Presenza Email nel Database [D]

- Email nuova (Non presente) [propertyNewEmail]
- Email duplicata (Già presente) [error]

Parametro: Codice Fiscale [CF]

- $^{\wedge}[A-Z]\{6\}[0-9]\{2\}[A-Z][0-9]\{2\}[A-Z][0-9]\{3\}[A-Z]\$$ Formato valido (16 caratteri alfanumerici corretti) [propertyCFOk]
- Formato errato (lunghezza o caratteri errati) [error]

Parametro: NomeUtente [NU]

- Lunghezza >8 and <20 [propertyLengthNUOk]
- Lunghezza <8 or >20 [error]

Parametro: Password [PW]

- Lunghezza >8 and <20 [propertyLengthPWOk]

- Lunghezza <8 or >20 [error]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_REG_1	F:2	Inserimento email con formato errato (es. senza '@').	Fail (Errore: "Formato email non valido")
TC_REG_2	F:1, D:2	Email formalmente valida ma già presente nel Database.	Fail (Errore: "Email già registrata")
TC_REG_3	CF:2	Codice Fiscale errato (es. lunghezza diversa da 16 o caratteri invalidi).	Fail (Errore: "Codice Fiscale non valido")
TC_REG_4	NU:2	Nome Utente fuori range (minore di 8 o maggiore di 20 caratteri).	Fail (Errore: "Lunghezza Nome Utente non valida")
TC_REG_5	PW:2	Password fuori range (minore di 8 o maggiore di 20 caratteri).	Fail (Errore: "Lunghezza Password non valida")
TC_REG_6	F:1, D:1, CF:1, NU:1, PW:1	Tutti i dati rispettano formati, lunghezze e unicità.	Pass (Registrazione completata con successo)

9.4 Funzionalità: Visualizzazione Referto (UC6)

L'obiettivo è verificare che il sistema gestisca correttamente sia la presenza che l'assenza del referto, mostrando all'utente le informazioni adeguate.

Parametro: Stato Referto [R]

- Referto presente [propertyRefertoOk]
- Referto assente (non ancora caricato) [propertyRefertoMissing]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_VIS_1	R:2	Utente accede a visita senza referto.	Pass (Viene mostrata la pagina: "Referto non ancora disponibile")
TC_VIS_2	R:1	Utente visualizza referto esistente.	Pass (Visualizzazione Dettagli)

9.5 Funzionalità: Modifica Prenotazione – Segreteria (UC4)

Si verifica che un operatore di Segreteria possa modificare data e ora di una prenotazione esistente, garantendo che l'operazione sia permessa solo su visite future e verso orari disponibili.

Parametro: Prenotazione Originale [ID]

- Esistente e Modificabile (Data futura) [propertyIdOk]
- Non esistente o Già Conclusa/Passata [error]

Parametro: Disponibilità Nuovo Slot [S]

- Slot Libero [propertySlotFree]
- Slot Occupato (Già prenotato da altri) [error]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_MOD_1	ID:2	Tentativo di modifica di una visita passata o con ID non valido.	Fail (Errore: "Prenotazione non modificabile")
TC_MOD_2	ID:1, S:2	Tentativo di spostamento su uno slot orario già occupato da un altro paziente.	Fail (Errore: "Orario non disponibile")